

# TP06 — API, mass assignment, quotas, proxy et conteneurs

Séances : S10–S11 · Durée : 180 min · Compétences : C08, C09 · Niveau : intermédiaire avancé

## Cadre, objectifs et prérequis

Scope : ShopLab local uniquement. Vous inventoriez OpenAPI, testez les propriétés inattendues et quotas, vérifiez les erreurs/logs, puis auditez proxy, forwarded headers, TLS et isolation conteneur. TP03–TP05 requis.

## Livrables et critères

Inventaire d'endpoints classé par actif/authz, preuve de mass assignment avant/après, série de requêtes produisant réellement un 429, baseline proxy/conteneurs et propositions priorisées. Réussite : propriété `role` ignorée/refusée en corrected, schéma fermé, quota mesurable, headers corrects, ports localhost, rootfs read-only/capabilities minimales.

## Mise en place (15 min)

```
cd labs/shoplab
./scripts/reset.sh
mkdir -p preuves/TP06
base=http://127.0.0.1:${SHOPLAB_HTTP_PORT:-8080}
curl -sS "$base/openapi.json" > preuves/TP06/openapi.json
```

## A — Inventaire API et erreurs (30 min)

À partir d'OpenAPI et non d'une exploration hors scope, relevez méthode, route, entrée, authentification, objet et erreurs attendues. Testez JSON malformé, mauvais type, champ inconnu et ressource absente. Vérifiez que les erreurs n'exposent ni requête SQL, stack trace, token ou chemin hôte.

## B — Mass assignment (35 min)

Connectez Alice. Envoyez à `PATCH /api/users/me { "display_name":"Alice TP06", "role":"admin", "internal_note":"x" }`. En vulnerable, documentez la propriété indûment acceptée sans exploiter le nouveau rôle. Reset immédiat. En corrected, attendu 422 ou liste `updated` limitée à `display_name`; relisez ensuite le profil et testez un champ imbriqué.

## C — Quota et journal (25 min)

Exécutez le script `scripts/exercise-rate-limit.sh` qui utilise un compte synthétique unique, s'arrête après le premier 429 et affiche un résumé PASS/WARN/FAIL. Associez le `X-Correlation-ID` au journal. Expliquez clé de quota, fenêtre, réponse `Retry-After`, risques de DoS et faux positifs. Un quota uniquement en mémoire n'est pas distribué.

## D — Proxy/TLS/forwarded headers (35 min)

```
curl -skI https://127.0.0.1:${SHOPLAB_HTTPS_PORT:-8443}/
docker compose --profile core config
docker compose --profile core exec -T api id
```

Vérifiez CSP, HSTS (seulement HTTPS), `X-Content-Type-Options`, permissions, ports liés à loopback, réseau core interne, absence de Docker socket, `read_only`, `cap_drop`, secrets synthétiques. Injectez un `X-Forwarded-For` factice et vérifiez qu'il n'accorde aucun privilège; documentez la liste de proxies de confiance requise en production.

## E — Retest (20 min)

Lancez `verify-lab.sh api` puis `verify-lab.sh hardening`. Chaque assertion doit indiquer contrôle, résultat observé et limite. Comparez code de l'application et configuration : le proxy ne corrige pas BOLA/mass assignment.

## Reset/cleanup (20 min)

```
./scripts/reset.sh
./scripts/health-check.sh core
./scripts/cleanup.sh
docker compose --profile core ps --all
```

## Dépannage et plateformes

Un 429 absent : vérifiez le mode corrected et utilisez le script (fenêtre/compte constants). HSTS n'est pas attendu sur HTTP. Rootless Docker peut afficher un UID numérique : jugez surtout non-root et capacités. Docker Desktop/WSL2 virtualisent le réseau; les assertions restent faites depuis l'hôte sur `127.0.0.1`.

## Barème /20

| Critère               | Insuffisant      | Conforme                    | Maîtrisé                   | Pts |
|-----------------------|------------------|-----------------------------|----------------------------|-----|
| API/schema            | OpenAPI copié    | inventaire + erreurs        | frontières objet/propriété | 5   |
| Mass assignment/quota | affirmation      | avant/après + vrai 429      | limites distribuées        | 6   |
| Infra                 | checklist vague  | preuves TLS/proxy/container | trust proxy argumenté      | 6   |
| Cycle/preuves         | résidus/sensible | reset/cleanup expurgé       | reproductible              | 3   |